DATEN → INFORMATION → WISSEN → HANDLUNG

Stufe	Bedeutung	Beispiel
Daten	Zeichen ohne Bedeutung	000000111 = 7
Information	Daten + Kontext	„Es sind 7 °C.“
Wissen	Informationen verknüpft	„7 °C ist in Deutschland kalt.“
Handlung	folgt aus Wissen	Jacke anziehen

Gleiche Daten, anderer Kontext → andere Information: 7 °C in Deutschland (kalt), in Kanada (warm), 7 °F in den USA (sehr kalt, etwa −14 °C).

DREI ASPEKTE VON INFORMATION

Aspekt	Frage	Ampel
syntaktisch	Form, Datentyp?	oberes Licht leuchtet rot
semantisch	Bedeutung?	„Halt.“
pragmatisch	Nutzen, was tun?	bremsen

- **EVA-Prinzip:** Eingabe → Verarbeitung → Ausgabe.
- Binär: 0 = keine Spannung, 1 = Spannung. 1 Stelle = **Bit**, 8 Bit = **Byte**.

INFORMATIONSTHEORIE (1948)

Quelle → Sender → Kanal (mit Störungen) → Empfänger → Ziel.

Redundanz: der eigentlich überflüssige Teil einer Nachricht („M wie Martha“).

Ziel	Redundanz	Beispiel
Kompression	entfernen	RLE: AAAABBBCC → 4A3B2C
Fehlererkennung	hinzufügen	Paritätsbit, Hamming-Code

RLE (Lauflängenkodierung): je Folge gleicher Zeichen Anzahl + Zeichen. WWWWWBBBWW → **5W3B2W**.

DIGITALISIERUNG: DREI SICHTWEISEN

Sicht	Kernaussage
betriebswirtschaftlich	Daten und Algorithmen für neue/bessere Prozesse, Produkte, Geschäftsmodelle; analoge Arbeit durch digitales Modell ersetzt; Querschnittstechnologie
gesellschaftlich	Lebens- und Arbeitswelten auf digitale Ebene übertragen, oft mit wirtschaftlichen Umbrüchen („Disruption“)
technisch	Elektrotechnik: analoge Signale → **zeit- und wertediskrete**, meist binäre Signale, um sie elektronisch zu übertragen, speichern, verarbeiten

- **zeitdiskret:** nur in festen Abständen messen (z. B. 44 100-mal pro Sekunde).
- **wertediskret:** jeden Messwert auf eine von endlich vielen Stufen runden (16 Bit = 65 536 Stufen).

PHASEN DER DIGITALISIERUNG IM UNTERNEHMEN

Phase	Was wird digitalisiert?	Beispiel
1	einzelne Funktionen	Rechnungen automatisch erstellen
2	ganze Funktionsbereiche	Rechnungswesen
3	integrierte Prozesse über Abteilungen	Auftragsbearbeitung
4	1:1-Verbindung mit einem Partner	lieferantengeführte Bestände
heute	m:n-Vernetzung bis zum Endkunden	B2B-/B2C-Marktplätze, Produktion „on demand“

Reichweite wächst: Funktion → Bereich → Prozess → 1 Partner → alle Partner.

ROLLE UND NUTZEN DER IT

- Vor dem Internet: „EDV-Abteilung“ für Routinearbeit, nicht strategisch. Danach: Teil des Kerngeschäfts (Shopsysteme, CRM, digitale Produkte).

Stufe	Bedeutung	Onlineshop
1. Automatisierung	IT erledigt bisherige Abläufe, schneller, einheitlich	Rechnungen automatisch
2. Informatisierung	entstandene Daten auswerten für bessere Entscheidungen (Data Warehouse)	Sortiment planen
3. Transformation	völlig neue Geschäftsprozesse	Fertigung erst nach Bestellung

Ökonomische Folgen: digitale Kopie kostet fast nichts · Plattformen ohne eigene Güter (Uber, Airbnb) · Long Tail (viele Nischenartikel) · Freemium (Gratisversion + Abo) · sinkende Eintrittsbarrieren.

TYPISCHE FEHLER

- Daten und Information gleichsetzen. Erklären mit dem 7-Grad-Beispiel.
- Paritätsbit macht die Nachricht kürzer: falsch, es fügt Redundanz hinzu.
- Technische Sicht als „kontinuierlich“ beschreiben. Digital heißt diskret.
- Phasen der Digitalisierung (wirtschaftlich) mit den fünf Epochen der IT-Infrastruktur (Kapitel 2) verwechseln.
- Daten auswerten als Automatisierung einordnen. Das ist Informatisierung.
- Transformation = schnellerer alter Ablauf: falsch, es ist ein neues Geschäftsmodell.
- Bei „Nennen“ lange erklären. Stichpunkte reichen.

SIEBEN HAUPTKOMPONENTEN

Nr	Komponente	Beispiel
1	Hardwareplattformen	Server, PCs
2	Betriebssystem-Plattformen	Windows, Linux
3	Datenverwaltung und -speicherung	Datenbankmanagementsystem
4	Netzwerke und Telekommunikation	LAN
5	Anwendungssysteme	ERP (SAP), CRM
6	Internet (als Plattform)	Webdienste
7	Beratungsdienste und Systemintegratoren	Firmen, die alles einrichten

Antwort auf „Welche Komponenten finden Sie in einem Unternehmen?“. Beim Nennen nachzählen: sieben.

FÜNF EPOCHEN DER IT-INFRASTRUKTUR

Epoche	Name	Merkmal
1	Groß- und Minirechner	ein Großrechner bedient viele Terminals; Hard- und Software meist von einem Anbieter
2	Personal Computing	IBM-PC, „Wintel“-PC als Standard; erst Einzelplätze, ab ca. 1990 vernetzt
3	Client-Server	Clients mit leistungsstarken Servern vernetzt
4	internetbasierte Informationssysteme	Netze über TCP/IP verbunden; Kunden und Lieferanten über Internet angebunden
5	Cloud Computing	Programme und Daten von externen Anbietern über das Internet

- Epochen **überlagern** sich: Großrechner gibt es bis heute.
- Jahreszahlen nicht prüfungsrelevant, Reihenfolge und je ein Merkmal schon.

CLIENT-SERVER

- Der Client fragt, der Server antwortet.
- **Client:** Eingabestelle des Benutzers (Laptop).
- **Server:** verarbeitet und speichert gemeinsam genutzte Daten, liefert Webseiten aus, verwaltet das Netz.
- Arbeit wird auf viele günstige Maschinen verteilt.

```
2-Tier:  Client <--> Server (Daten + Logik)
N-Tier:  Client <-- Internet --> Webserver <-->
Anwendungsserver <--> Datenbankserver
```

- **2-Tier:** Client spricht direkt mit einem Server. **N-Tier** (mehrschichtig): Arbeit auf mehrere Server-Ebenen verteilt, das Internet ist nur der Zugang zum Webserver. Meist genutzt.
- Skizze: alle Kästen beschriften, Pfeile in beide Richtungen.

CLOUD COMPUTING

- Rechner und Programme mieten statt selbst kaufen und betreiben; Bezug von externen Anbietern über das Internet, meist im Abo, oft per Browser.
- **virtuell:** Rechner existiert nur als Software auf einem großen echten Rechner.
- **skalierbar:** per Klick mehr oder weniger mieten.
- Beispiel: New York Times wandelte mit gemieteten Amazon-Rechnern ca. 11 Mio. Artikel in 24 h in PDF um.
- AWS entstand, weil Amazon Rechenleistung fürs Weihnachtsgeschäft brauchte und sie den Rest des Jahres vermieten wollte.
- **Hyperscaler:** AWS, Microsoft Azure, Google Cloud, STACKIT.

IAAS, PAAS, SAAS

Modell	Du mietest	Du machst selbst	Beis
IaaS (Infrastructure as a Service)	Rechenleistung, Speicher, Netz als VMs	Betriebssystem + Anwendungen	virtu Serv AWS
PaaS (Platform as a Service)	zusätzlich Laufzeitumgebung, Datenbank	nur deine Anwendung	Goo Eng BTP
SaaS (Software as a Service)	fertige Anwendung im Browser	nichts, nur nutzen	Sale Micı 365

IT-LANDSCHAFT STRUKTURIEREN

- nach **Schichten:** die sieben Komponenten
- nach **Architektur:** zentral (Großrechner), Client-Server (2-Tier/N-Tier), verteilt übers Internet
- nach **Betriebsmodell:** eigener Betrieb oder Cloud (IaaS, PaaS, SaaS)

TYPISCHE FEHLER

- „Internet als Plattform“ und „Beratungsdienste und Systemintegratoren“ vergessen.
- Glauben, mit der Cloud seien Großrechner verschwunden. Die Epochen überlagern sich.
- Epochen mit den Phasen der Digitalisierung (Kapitel 1) verwechseln.
- Client und Server vertauschen: Der Server hält die gemeinsamen Daten.
- Skizze mit Pfeilen nur in eine Richtung.
- IaaS und PaaS verwechseln: Installierst du das Betriebssystem selbst, ist es IaaS.

ORIGINAL (1965)

- Die **Anzahl** der Transistoren (Komponenten) pro Chip **verdoppelt sich jährlich**; später korrigiert auf **alle zwei Jahre**.
- Grund für den Preiseffekt: Fläche ist der wichtigste Kostenfaktor. Mehr Transistoren pro Fläche = mehr Leistung fürs gleiche Geld.

DREI INTERPRETATIONEN (NICHT VON MOORE)

1. Leistung von **Mikroprozessoren** verdoppelt sich alle 18 Monate.
2. Leistung von **Computern** verdoppelt sich alle 18 Monate.
3. **Preis** für rechnergestützte Informationsverarbeitung **halbiert** sich alle 18 Monate.

	Was ändert sich?	Zeitraum
Original	Anzahl Transistoren pro Chip ×2	jährlich, später 2 Jahre
Interpretationen	Leistung ×2 bzw. Preis ÷2	18 Monate

MIT VERDOPPLUNGEN RECHNEN

Faktor = 2 hoch (t / T). t = gesamter Zeitraum, T = Zeit für eine Verdopplung, t / T = Anzahl Verdopplungen.

1. t / T = Anzahl der Verdopplungen.
2. 2 hoch diese Anzahl = Wachstumsfaktor.
3. Startwert × Faktor.

Verdopplungen	0	1	2	3	4	5	6	10
Faktor	1	2	4	8	16	32	64	1 024

- 1 Mrd. Transistoren, 10 Jahre, T = 2: 10 / 2 = 5 → 2^5 = 32 → **32 Mrd.**
- 12 Jahre, T = 2: 6 Verdopplungen → 2^6 = 64 → **64 Mrd.**
- 6 Jahre, T = 18 Monate: 6 / 1,5 = 4 → Faktor **16**.

GRENZEN UND AUSWEGE

- **Grenzen:** Kosten pro Transistor steigen bei immer kleineren Strukturen wieder; Energiebedarf und Kühlung wachsen mit der Rechenleistung.
- **Auswege:** neue Materialien und Lithographie-Verfahren, 3D-Chipdesign, Quantencomputer, Skalierung über viele GPUs, Cluster, Rechenzentren.
- Größenordnung: aktueller KI-Chip ca. 208 Mrd. Transistoren.

MULTIPLE CHOICE: BEWERTUNG

- Mindestens eine Antwort ist immer richtig.
- Jede richtige Entscheidung (angekreuzt **oder** nicht angekreuzt) +½, jede falsche −½. Negative Summe = 0. Nichts ankreuzen = 0.
- 4 Optionen, alles richtig entschieden: 4 × ½ = **2 P.** Eine falsche zusätzlich angekreuzt: 3 × ½ − ½ = **1 P.**
- Vorgehen: jede Option Wort für Wort lesen, das eine falsche Wort suchen (verdreifacht/verdoppelt, Speicher/Leistung, Größe/Anzahl), einzeln entscheiden.

FÜNF TREIBER DES FORTSCHRITTS

1. Mooresches Gesetz und Mikroprozessorleistung
2. Digitale Massenspeicher: Kapazität steigt, Preise fallen; Datenmenge verdoppelt sich grob jedes Jahr
3. Gesetz von Metcalfe: Nutzen eines Netzes wächst überproportional mit der Teilnehmerzahl
4. Sinkende Kommunikationskosten und Wachstum des Internets
5. Standardisierung (ASCII, Ethernet, TCP/IP, HTTP): kompatible Geräte, sinkende Preise

METCALFE RECHNEN

Verbindungen = n · (n − 1). n = Teilnehmerzahl; jeder erreicht die n − 1 anderen (A→B und B→A getrennt gezählt).

n	8	9	10	11	20
n · (n − 1)	56	72	90	110	380

Doppelt so viele Teilnehmer (10 → 20) → gut viermal so viele Verbindungen.

TYPISCHE FEHLER

- „Original: Leistung verdoppelt sich alle 18 Monate." Das ist eine Interpretation.
- Verdreifacht statt verdoppelt.
- „Speicherkapazität verdoppelt sich": keine Interpretation.
- „Größe eines Transistors halbiert sich": es geht um die Anzahl.
- „Kosten verdoppeln sich": sie halbieren sich.
- „Taktfrequenz verdoppelt sich": keine Interpretation.
- 5 Verdopplungen als 5 × 2 = 10 rechnen statt 2^5 = 32.
- Bei „zwei Grenzen und zwei Auswege" nur Auswege nennen.
- Metcalfe „exponentiell" nachrechnen wollen: die Formel n · (n − 1) wächst quadratisch.

AUSWENDIG KÖNNEN

Basis	Stellenwerte (von rechts)
2	1 · 2 · 4 · 8 · 16 · 32 · 64 · 128 · 256 · 512 · 1024
8	1 · 8 · 64 · 512
16	1 · 16 · 256 · 4096

Hexziffern: A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

UMRECHNEN

Basis b → dezimal: jede Ziffer mal ihren Stellenwert, alles addieren. $1011_2 = 8 + 0 + 2 + 1 =$ **11**. $2F_{16} = 2 \cdot 16 + 15 =$ **47**.

Dezimal → Basis b (Restwertmethode): immer wieder durch b teilen, Reste **von unten nach oben** lesen.

13 : 2	Rest
13 → 6	1
6 → 3	0
3 → 1	1
1 → 0	1

13 = $\mathbf{1101_2}$

Binär ↔ Oktal / Hex: Gruppen bilden, **immer von rechts**. 3 Bit = 1 Oktalziffer, 4 Bit = 1 Hexziffer. $0110\ 0100_2 = \mathbf{64_{16}}$. $001\ 100\ 100_2 = \mathbf{144_8}$. Beides = 100.

BINÄR ADDIEREN

- 0 + 0 = 0 · 0 + 1 = 1 · 1 + 1 = 0, Übertrag 1 · 1 + 1 + 1 = 1, Übertrag 1
- Übertrag aus der Spalte rechts immer mitzählen.

NEGATIVE ZAHLEN: ZWEIERKOMPLEMENT (FESTE BITBREITE!)

1. Positive Zahl in n Bit schreiben: 25 = 0001 1001
2. Alle Bits umdrehen: 1110 0110
3. Plus 1: **1110 0111 = −25**

Ablesen: Ganz links eine 1 → negativ. Umdrehen + 1 ergibt den Betrag, Minus davor. 8 Bit: Bereich **−128 … +127**. Allgemein $-2^{n-1} \ldots 2^{n-1} - 1$.

SUBTRAHIEREN

a − b = a + (Zweierkomplement von **b**). Übertrag links aus den n Bit fällt weg.

30 − 45: 0001 1110 + 1101 0011 (= −45) = 1111 0001 = **−15** ✓

ÜBERLAUF

Das Ergebnis passt nicht in den Bereich. Erkennbar am Vorzeichen:

- plus + plus = minus → Überlauf
- minus + minus = plus → Überlauf
- plus + minus → nie ein Überlauf

127 + 127 = 254: in 8 Bit **ohne** Vorzeichen ok, im 8-Bit-Zweierkomplement Überlauf.

KOMMAZAHLEN

Nachkommateil fortlaufend **mal 2**, die Ziffer vor dem Komma ist das nächste Bit, von **oben nach unten** lesen. 0,625 · 2 = 1,25 → 1 · 0,25 · 2 = 0,5 → 0 · 0,5 · 2 = 1,0 → 1 ⇒ $\mathbf{0{,}101_2}$

TYPISCHE FEHLER

- Stellen von links zählen. Die letzte Stelle hat immer den Wert 1 (Position 0).
- Gruppen von links bilden statt von rechts.
- Restwertmethode von oben gelesen statt von unten.
- Beim Zweierkomplement die +1 vergessen, oder das Komplement von der falschen Zahl gebildet.
- 1110 0111 als 231 lesen, obwohl es eine Zahl mit Vorzeichen ist.
- Hinter dem Komma durch 2 teilen statt mal 2 nehmen.
- Probe vergessen: Ergebnis zurück ins Dezimalsystem rechnen.

GRUNDOPERATOREN

- Nur zwei Werte: 0 (falsch, aus), 1 (wahr, an).
- **AND** (A ∧ B, A · B): nur 1, wenn alle Eingänge 1. **OR** (A ∨ B, A + B): nur 0, wenn alle Eingänge 0. **NOT** (¬A): dreht um.
- Rangfolge: ¬ vor ∧ vor ∨. Im Zweifel Klammern.
- Transistoren in Serie = AND, parallel = OR.

SIEBEN GATTER

A	B	AND	OR	XOR	NAND	NOR	XNOR
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

XOR = „ungleich“, XNOR = „gleich“. NAND/NOR/XNOR = AND/OR/XOR umgedreht.

IEC 60617-12 (Kästchen mit Zeichen, Kreis am Ausgang = Negation):

AND	OR	NOT	XOR	NAND	NOR	XNOR
&	≥1	1 + Kreis	=1	& + Kreis	≥1 + Kreis	=1 + Kreis

RECHENGESETZE

Gesetz	UND	ODER
Kommutativ	a ∧ b = b ∧ a	a ∨ b = b ∨ a
Distributiv	a ∧ (b ∨ c) = (a ∧ b) ∨ (a ∧ c)	a ∨ (b ∧ c) = (a ∨ b) ∧ (a ∨ c)
Idempotenz	a ∧ a = a	a ∨ a = a
Neutralität	a ∧ 1 = a	a ∨ 0 = a
Extremal	a ∧ 0 = 0	a ∨ 1 = 1
Komplement	a ∧ ¬a = 0	a ∨ ¬a = 1
Doppelnegation	¬¬a = a	
De Morgan	¬(a ∧ b) = ¬a ∨ ¬b	¬(a ∨ b) = ¬a ∧ ¬b
Absorption	a ∨ (a ∧ b) = a	a ∧ (a ∨ b) = a

De Morgan: Negation über die Klammer ziehen → ∧ wird ∨ (und umgekehrt), jede Variable negiert. ¬(¬A ∨ B) = ¬¬A ∧ ¬B = **A ∧ ¬B**.

TEXT → TABELLE → DNF → SCHALTUNG

Tabelle:

1. n Eingänge → 2^n Zeilen.
2. Eingänge binär von 000 bis 111 hochzählen.
3. Je Zeile Ausgang 0 oder 1 bestimmen.

DNF (disjunktive Normalform):

1. Nur Zeilen mit Ausgang 1 nehmen.
2. Je 1-Zeile ein UND-Term: 1 → Variable, 0 → ¬Variable (101 → S ∧ ¬T ∧ F).
3. Alle Terme mit ∨ verbinden.

Unvereinfachte DNF ist korrekt, solange nicht „vereinfachen“ verlangt ist.

Vereinfachen (Kernregel): Zwei Terme, die sich nur in **einer** Variablen unterscheiden → diese fällt weg. (A ∧ B) ∨ (A ∧ ¬B) = A ∧ (B ∨ ¬B) = A ∧ 1 = A

1. Paare suchen, die sich in genau einer Variablen unterscheiden.
2. Gemeinsames ausklammern, X ∨ ¬X = 1 streichen.
3. Wiederholen. Ein Term darf mehrfach benutzt werden (Idempotenz).
4. Probe mit einer 1-Zeile und einer 0-Zeile.

Beispiel Alarmanlage (Y, wenn scharf S und Tür T oder Fenster F offen): 1-Zeilen STF = 101, 110, 111. Y = (S∧¬T∧F) ∨ (S∧T∧¬F) ∨ (S∧T∧F) → S∧F ∨ S∧T = **S ∧ (T ∨ F)**.

Muster: (¬A∧B) ∨ (A∧¬B) = XOR. (¬A∧¬B) ∨ (A∧B) = XNOR.

Schaltung: von innen nach außen, jede Operation ein Gatter. DNF direkt: NOT für negierte Eingänge, je Term ein UND, am Ende ein ODER.

Analyse (Schaltung → Tabelle): je Gatter eine Hilfsspalte, Zeile für Zeile von innen nach außen rechnen.

NAND-SUBSTITUTION

NAND ist ein Universalgatter:

- NOT A = NAND(A, A)
- A AND B = NAND, danach NAND als NOT (2 Gatter)
- A OR B = NAND(¬A, ¬B), 3 Gatter (De Morgan: ¬(¬A ∧ ¬B) = A ∨ B)

HALBADDIERER

A	B	Summe s	Übertrag c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- Halbaddierer: s = **XOR**, c = **AND**.
- Volladdierer: zusätzlich Übertrag der vorigen Stelle; 2 Halbaddierer + 1 ODER. Mehrere hintereinander = Addierwerk.

TYPISCHE FEHLER

- ¬(A ∧ B) = ¬A ∧ ¬B. Falsch: ∧ muss zu ∨ werden.
- OR mit XOR verwechseln: bei A = B = 1 ist OR 1, XOR 0.
- 0-Zeilen in die DNF aufnehmen.
- Terme zusammenfassen, die sich in zwei Variablen unterscheiden.
- Probe mit einer Zeile vergessen.
- Rangfolge ignorieren (¬ vor ∧ vor ∨).
- AND aus nur einem NAND bauen. Es braucht ein zweites als NOT.
- Summe beim Halbaddierer als OR: bei 1 + 1 muss sie 0 sein.

ALGORITHMUS, PROGRAMM, PROZESS

- **Algorithmus:** Handlungsvorschrift, die ein Problem in **endlicher Zeit** löst.
- **Programm:** Umsetzung eines oder mehrerer Algorithmen in einer Programmiersprache; darf endlos laufen.
- **Prozess: laufendes** Programm mit Daten, Variablenwerten und Verwaltungsinformationen des Betriebssystems.
- Uhrzeit jede Sekunde anzeigen: Programm, aber kein Algorithmus (endet nie). Darstellung: Struktogramm, UML-Aktivitätsdiagramm.

MASCHINENSPRACHE → ASSEMBLER → HOCHSPRACHE

- **Maschinensprache:** binäre Befehle, je Prozessortyp anders, kaum lesbar.
- **Assembler:** kurze Namen für Befehle (ADD) und Adressen; 1 Assemblerbefehl = 1 Maschinenbefehl.
- **Hochsprache** (Python, Java, C++): für Menschen lesbar, braucht Compiler oder Interpreter.
- **Register:** Speicher direkt im Prozessor. Prinzip: laden → rechnen → zurückschreiben.

SYNTAX- UND SEMANTIKFEHLER

	Syntaxfehler	Semantikfehler
Was?	falsche Schreibweise, z. B. fehlende Klammer	richtig geschrieben, falsche Bedeutung
Folge	Compiler übersetzt nicht	läuft, falsches Ergebnis, keine Meldung

Typfalle (Java): int / int ergibt int, der Rest wird abgeschnitten.

```
double res  = 10 / 3;           // 3.0
double res2 = (double) 10 / 3;  // 3.333…
```

VARIABLEN UND DATENTYPEN (JAVA)

Variable = Speicherplatz mit Namen und Typ. Erst lesbar, wenn ein Wert zugewiesen wurde.

- Deklaration `int zaehler;` · mit Initialisierung `int summe = 0;` · Zuweisung `summe = summe + 5;`

Typ	Größe	Bereich
byte	8 Bit	−128 … 127
short	16 Bit	−32 768 … 32 767
int	32 Bit	ca. ±2,1 Mrd.
long	64 Bit	ca. $\pm 9{,}2 \cdot 10^{18}$
float / double	32 / 64 Bit	Gleitkomma (IEEE 754)
char	16 Bit	ein Unicode-Zeichen
boolean	laut Folie 1 Bit	true, false

COMPILER UND INTERPRETER

	Compiler	Interpreter
Wann?	vorab, ganzes Programm	zur Laufzeit, Befehl für Befehl
Ergebnis	Maschinenprogramm für bestimmte Hardware	direkte Ausführung

Zwischenschritt möglich: maschinenunabhängiger **Bytecode** (Java, Python).

Compilerphasen: 1. Scanner (lexikalische Analyse, Grundsymbole) → 2. Parser (syntaktische Analyse, Syntaxbaum) → 3. semantische Analyse + Optimierung → 4. Codeerzeugung + -optimierung → Maschinenprogramm

LINKER, LADER, DEBUGGER

- **Linker:** verbindet einzeln übersetzte Module zu einem lauffähigen Programm.
- **statisch:** alles in eine Datei gebunden. + alles sicher vorhanden, – große Datei.
- **dynamisch:** Funktionen erst zur Laufzeit gesucht (DLLs, Shared Libraries). + gemeinsame Speichernutzung, Bibliotheken austauschbar, – fehlt eine Funktion → Abbruch.
- **Lader:** bringt das Programm in den Arbeitsspeicher, addiert den Startwert (Offset) zu allen Adressen. Laden an 0x2000: Sprungziel 0x0040 → **0x2040**.
- **Debugger:** Programm Schritt für Schritt, zeigt Speicher, Register, Variablen. **IDE** (z. B. Eclipse): Editor + Compiler + Debugger.

REKURSION

- **Teile und Herrsche** = Prinzip: Aufgabe in gleichartige kleinere Teilaufgaben zerlegen.
- **Rekursion** = Technik: Funktion ruft sich selbst mit kleinerer Eingabe auf. Braucht eine **Abbruchbedingung**.

```
long fakrek(int n) {
    if (n == 1 || n == 0) return 1;  // Abbruch
    return n * fakrek(n - 1);        // Selbstaufruf
}
```

fakrek(4): 4 · fakrek(3) → 3 · fakrek(2) → 2 · fakrek(1) = 1 (Abbruch). Rückweg: 2 · 1 = 2 → 3 · 2 = 6 → 4 · 6 = **24**.

fakrek(5) = **120**, 5 Aufrufe. Offene Aufrufe liegen auf dem Aufrufstapel (Stack).

TYPISCHE FEHLER

- „Ein Programm muss terminieren." Nur der Algorithmus muss endlich sein.
- Assembler als Hochsprache bezeichnen.
- 7 / 2 in Java = 3,5 erwarten. Ergebnis 3, das ist ein Semantikfehler.
- Linker und Lader verwechseln: Linker baut zusammen, Lader lädt in den Speicher.
- Teile und Herrsche mit Rekursion gleichsetzen (Prinzip vs. Technik).
- Abbruchbedingung vergessen → endlose Aufrufe, Aufrufstapel läuft über.

Datenstrukturen: Array, Liste, Stack, Queue, Baum

BEGRIFFE

Datenstruktur: legt mehrere Daten, auch verschiedener Typen, gemeinsam im Speicher ab (String, Struktur „Kunde“, Tabelle). int = Datentyp, Array von int = Datenstruktur.

ARRAY

- Fortlaufender **Index ab 0**, **feste Länge**, direkter Zugriff über a[i].
- + sehr schneller Zugriff. – Einfügen/Entfernen nur durch Verschieben/Umkopieren; voll → neues, größeres Array.

VERKETTETE LISTE

- Kein Index, nur der **Kopf** ist bekannt. Jedes Element: Daten + Zeiger **next** (Adresse des Nachfolgers), letztes → null.
- + Einfügen/Löschen nur Zeiger umhängen, Länge flexibel. – Lesen: vom Kopf aus durchlaufen.

X hinter A einfügen (Reihenfolge!):

1. X.next = A.next
2. A.next = X

B löschen: Vorgänger A suchen, A.next = B.next.

Doppelt verkettet: prev und next; beide Richtungen, Vorgänger beim Löschen nicht suchen; dafür mehr Speicher.

Sortierte Liste: vom Kopf laufen bis zum ersten größeren Element, davor einhängen. 10 → 19 → 36, 25 einfügen: 36 > 25 → stopp → 10 → 19 → 25 → 36.

STACK UND QUEUE

	Stack	Queue
Prinzip	**LIFO** (Last In, First Out)	**FIFO** (First In, First Out)
Rein	push(x): oben drauf	put(x)/enqueue: hinten an
Raus	pop(): oberstes nehmen	get()/dequeue: vorderstes nehmen
Lesen	top(): oberstes lesen, bleibt	–
Fehler	pop/top auf leerem Stack	get auf leerer Queue
Bau	Array + Index oder Liste	Ringpuffer im Array

Durchspielen: Zustand nach jeder Operation notieren, Ausgaben mitschreiben.

- Stack: push 4, 8, 2, pop, push 5, top, pop, pop → Ausgaben **2, 5, 5, 8**, Rest [4].
- Queue: put 4, 8, 2, get, put 5, get, get → Ausgaben **4, 8, 2**, Rest [5].

BAUM: BEGRIFFE

- **Wurzel:** Ursprung, oben, Ebene 0. Jeder andere Knoten hat genau einen Vorgänger.
- **Blatt:** ohne Nachfolger. **Innerer Knoten:** mindestens ein Nachfolger.
- **Ebene:** Kanten von der Wurzel bis zum Knoten. **Höhe:** höchste Ebene. **Pfadlänge:** Summe der Ebenen aller Knoten.

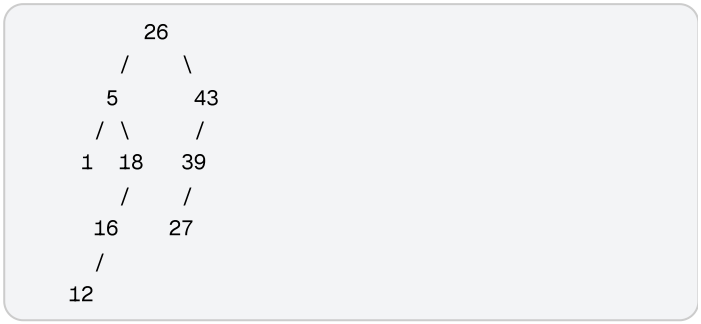
GEORDNETER BINÄRBAUM

Höchstens 2 Kinder; links nur kleinere, rechts nur größere Werte.

Einfügen/Suchen:

1. An der Wurzel beginnen.
2. Kleiner → links, sonst rechts.
3. Bis ein freier Platz kommt, dort als Blatt anhängen.

[26, 5, 18, 16, 1, 43, 39, 27, 12] in dieser Reihenfolge:



Höhe 4, Blätter 1, 12, 27. **Probe:** links – Knoten – rechts gelesen ergibt 1, 5, 12, 16, 18, 26, 27, 39, 43.

Löschen:

1. Blatt: entfernen.
2. Ein Kind: Kind rückt mit Unterbaum nach (18 löschen → 16 mit 12 rückt hoch).
3. Zwei Kinder: ersetzen durch **größten Wert links** (1× links, dann ganz rechts) oder **kleinsten Wert rechts** (1× rechts, dann ganz links); Ersatz an alter Stelle nach Fall 1/2 entfernen. Variante angeben.

- **voll:** jeder Knoten 0 oder 2 Kinder. **vollständig:** voll + alle Blätter auf der letzten Ebene.
- **balanciert:** Blätter auf letzter oder vorletzter Ebene; Tiefe ≈ $\log_2(n + 1)$, 1 000 Knoten ≈ 10 Ebenen.
- **degeneriert:** jeder Knoten höchstens 1 Kind = Liste (sortierte Eingabe 1, 2, 3, 4).

VERGLEICH

Struktur	Zugriff	+ / –
Array	O(1)	schnell / starr
Liste	O(n)	flexibel / langsame Suche
sortierte Liste	O(n)	sortiert / Einfügen teurer
Stack, Queue	nur oben/vorne	einfach / kein freier Zugriff
Binärbaum	balanciert O(log n)	schnell / kann degenerieren

TYPISCHE FEHLER

- a[10] bei 10 Elementen: Index endet bei 9.
- Beim Einfügen zuerst A.next umbiegen → Rest der Liste verloren.
- LIFO und FIFO vertauschen; top() entfernt nichts, pop() schon.
- Fehlerfall (leere Struktur) vergessen.
- Ebenen ab 1 zählen statt ab 0.
- Zahlen nicht in der gegebenen Reihenfolge einfügen.
- Bei zwei Kindern einfach das linke Kind hochziehen.
- Probe vergessen: von links nach rechts muss die Folge sortiert sein.

O-NOTATION

Beschreibt, wie der Aufwand wächst, wenn die Datenmenge n wächst. Konstante Faktoren fallen weg. **log n:** Wie oft kann man n halbieren, bis 1 übrig ist? (n = 1 000 → etwa 10)

Klasse	Name	Beispiel	n = 1 000	n = 1 000 000
O(1)	konstant	Zugriff auf a[5]	1	1
O(log n)	logarithmisch	binäre Suche	≈ 10	≈ 20
O(n)	linear	lineare Suche	1 000	1 000 000
O(n log n)	n log n	Merge Sort	≈ 10 000	≈ 20 000 000
O(n²)	quadratisch	Bubble Sort	1 000 000	10^{12}

- Schnell → langsam: O(1) < O(log n) < O(n) < O(n log n) < O(n²).
- n verdoppelt: O(1) gleich, O(log n) +1 Schritt, O(n) ×2, O(n²) ×4.

LINEARE SUCHE

- Alle Elemente der Reihe nach mit dem gesuchten Wert vergleichen.
- **Voraussetzung:** keine (auch unsortiert, auch Listen). **Aufwand:** O(n), schlechtester Fall n Vergleiche.

BINÄRE SUCHE

Voraussetzung: sortiert + Indexzugriff (Array). **Aufwand:** O(log n).

1. lo = erster Index, hi = letzter Index.
2. m = (lo + hi) / 2, abgerundet.
3. a[m] = gesucht → fertig.
4. Gesucht größer → lo = m + 1; kleiner → hi = m − 1.
5. Wiederholen. lo > hi → nicht vorhanden.

Suche 61 in [10, 19, 23, 25, 36, 61, 79, 81, 99] (Index 0–8):

Schritt	Bereich	m	a[m]	Entscheidung
1	0–8	4	36	61 > 36 → rechts
2	5–8	6	79	61 < 79 → links
3	5–5	5	61	gefunden

Höchstzahl Schritte: kleinstes k mit $2^k > n$.

n	15	100	500	1 000	1 000 000
k	4 ($2^4 = 16$)	7 (128)	9 (512)	10 (1 024)	20 (≈ 1,05 Mio.)

BUBBLE SORT

1. Von vorne je zwei Nachbarn vergleichen.
2. Falsch herum → tauschen.
3. Bis zum Ende des unsortierten Teils; das letzte Element ist fertig.
4. Neuer Durchlauf; Ende, wenn ein Durchlauf ohne Tausch bleibt.

[5, 1, 4, 2, 8]:

- Durchlauf 1: 5↔1, 5↔4, 5↔2 → [1, 4, 2, 5, 8] (8 fertig)
- Durchlauf 2: 4↔2 → [1, 2, 4, 5, 8]
- Durchlauf 3: kein Tausch → sortiert

Nach Durchlauf 1 steht das **größte** Element hinten. Höchstens n − 1 Durchläufe. **Voraussetzung:** keine. **Aufwand:** O(n²).

MERGE SORT

Teile und Herrsche, rekursiv.

1. Array in der Mitte teilen.
2. Weiter teilen, bis jedes Stück 1 Element hat (immer sortiert).
3. Je zwei sortierte Stücke mergen: vorderste Elemente vergleichen, kleineres übernehmen.
4. Bis nur ein Stück übrig ist.

```
Teilen:   [38, 27, 43]          [3, 9, 82, 10]
          [38] [27, 43]         [3, 9] [82, 10]
          [38] [27] [43]        [3] [9] [82] [10]
Mischen:  [27, 38, 43]          [3, 9, 10, 82]
          [3, 9, 10, 27, 38, 43, 82]
```

Letzter Merge: 27/3 → 3, 27/9 → 9, 27/10 → 10, 27/82 → 27, 38/82 → 38, 43/82 → 43, Rest 82 anhängen.

Voraussetzung: keine. **Aufwand:** O(n log n): ≈ $\log_2$ n Ebenen, je Ebene jedes Element einmal.

VERGLEICH

Verfahren	Voraussetzung	Prinzip	Aufwand
lineare Suche	keine	alle nacheinander	O(n)
binäre Suche	sortiert, Index	Mitte, Hälfte verwerfen	O(log n)
Suche im Binärbaum	geordneter Binärbaum	links kleiner, rechts größer	balanciert O n), degenerie O(n)
Bubble Sort	keine	Nachbarn tauschen	O(n²)
Merge Sort	keine	teilen, mischen	O(n log n)
Quicksort	keine	Pivot, kleiner/größer trennen, rekursiv	Mittel O(n lo schlimmster O(n²)
Selection Sort	keine	kleinstes suchen, nach vorne tauschen	O(n²)

TYPISCHE FEHLER

- Binäre Suche auf unsortierten Daten.
- Binäre Suche als O(n log n) angeben. Richtig: O(log n); O(n log n) ist Merge Sort.
- Bei gerader Anzahl die Mitte mal ab-, mal aufrunden. Bei einer Regel bleiben.
- „Nach Durchlauf 1 steht das kleinste vorne." Das größte steht hinten.
- Bubble Sort als O(n log n) angeben.
- Bei Merge Sort beim Teilen sortieren. Geteilt wird stumpf in der Mitte, sortiert beim Mischen.
- Lineare Suche als „schlecht" abtun: bei unsortierten Daten die einzige Möglichkeit.
- Matrixmultiplikation (n × n) unter O(n²) einordnen, wie auf der Folie. Das einfache Verfahren ist O(n³).

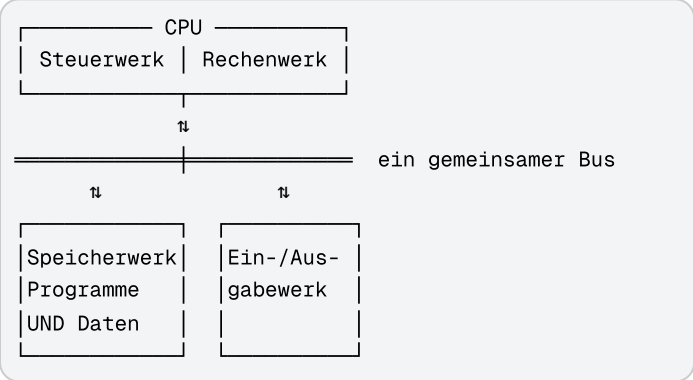
VON-NEUMANN-ARCHITEKTUR (1946)

- Daten **und** Programme binär im **gleichen Speicher**, Transport über den **gleichen Bus**.
- Folge: Programme wie Daten ladbar, änderbar, speicherbar → universell programmierbar (Stored-Program-Prinzip).
- Befehle der Reihe nach; Steuerung und Rechnen getrennt. **Bus:** Leitungsbündel für den Datenaustausch.

FÜNF KOMPONENTEN

Komponente	Aufgabe
Rechenwerk (ALU)	Rechenoperationen, logische Verknüpfungen
Steuerwerk (Control Unit)	interpretiert Befehle, regelt die Befehlsfolge
Bussystem	verbindet alles (Steuer-, Adress-, Datenbus)
Speicherwerk (RAM)	Programme **und** Daten
Ein-/Ausgabewerk	Tastatur, Bildschirm, andere Systeme

CPU = Steuerwerk + Rechenwerk (+ Register).



Skizze: 5 Kästen beschriftet · CPU umrahmt · „Programme und Daten“ am Speicher · **ein** Bus · Pfeile in beide Richtungen.

REGISTER (SCHNELLE SPEICHER IN DER CPU)

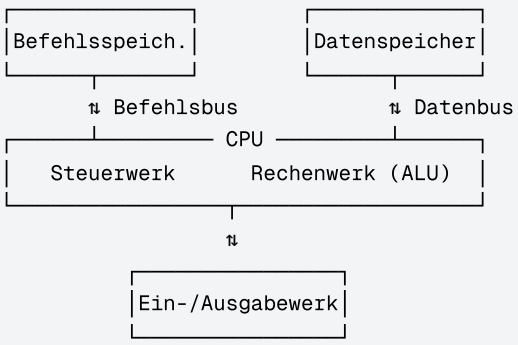
- **Akkumulator** (Rechenwerk): Operanden und Ergebnisse.
- **Befehlszähler** (Program Counter, Steuerwerk): **Adresse** des nächsten Befehls.
- **Befehlsregister** (Instruction Register, Steuerwerk): **aktueller Befehl** selbst.
- **Statusregister:** z. B. wahr/falsch nach einem Vergleich.

VON-NEUMANN-ZYKLUS (5 PHASEN)

1. **FETCH:** Befehl aus dem Speicher ins Befehlsregister laden.
2. **DECODE:** Steuerwerk entschlüsselt ihn in Schaltanweisungen.
3. **FETCH OPERANDS:** Operanden aus dem Speicher holen.
4. **EXECUTE:** Rechenwerk führt aus, schreibt ggf. Ergebnis in den Speicher.
5. **UPDATE INSTRUCTION POINTER:** Befehlszähler erhöhen oder bei Sprung neu setzen.

HARVARD

- **Getrennte Speicher** (eigene Adressen) **und getrennte Busse** für Befehle und Daten.
- **Einzelzyklus:** Befehl und Daten gleichzeitig holen → schneller, aber in der Regel teurer.
- Einsatz: Mikrocontroller, Signalverarbeitung, Rechner für maschinelles Lernen, zunehmend PCs und Server.



PRO UND CONTRA

Kriterium	Von Neumann	Harvard
Speicher / Bus	je einer, gemeinsam	getrennt
Ablauf	nacheinander, mind. 2 Takte	gleichzeitig, Einzelzyklus
Tempo	durch Flaschenhals begrenzt	schneller
Kosten	Pro: einfacher, billiger	Contra: teurer
Sicherheit	Contra: als Daten getarnter Code (Buffer Overflow)	Code und Daten getrennt
Einsatz	klassische PCs, Intel x86	Mikrocontroller, Signalverarbeitung, KI-Rechner

Von-Neumann-Flaschenhals: Code und Daten laufen nacheinander über den gemeinsamen Bus; der Engpass ist der **Bus**.

Prozessoren: Intel x86 = Von Neumann · AVR (Arduino) = Harvard · Apple A/M, ARM (Snapdragon) = **modifizierte Harvard**: zwischen CPU und Caches getrennt, Hauptspeicher gemeinsam.

CPU UND BUSSE

- Moderne CPUs schneller durch: mehrstufige Caches (L1–L3), Sprungvorhersage, spekulative Ausführung, Multicore.

Bus	überträgt
Steuerbus	**ob** gelesen/geschrieben wird
Adressbus	**wo**
Datenbus	**was**

Lesen: 1. Lesebefehl (Steuerbus) → 2. Adresse (Adressbus) → 3. Daten Zelle → CPU (Datenbus). Schreiben: in Schritt 3 Daten CPU → Zelle.

TYPISCHE FEHLER

- Bus in der Skizze vergessen.
- Befehlszähler (Adresse) mit Befehlsregister (Befehl) verwechseln.
- Nur „Fetch – Decode – Execute“ nennen statt fünf Phasen.
- Harvard nur mit „getrennte Speicher“ erklären (auch Busse!).
- „Flaschenhals, weil die ALU zu langsam ist“: Engpass ist der Bus.
- Adress- und Datenbus verwechseln.
- Pro und Contra nicht paarweise nennen.

SPEICHERARTEN (SCHNELL → LANGSAM)

Art	Eigenschaften
Register	in der CPU, extrem schnell, sehr klein, teuer
Cache (L1, L2, L3)	sehr schneller Zwischenspeicher für Daten und Programmteile, Teil der CPU, aus Kostengründen klein
Hauptspeicher (RAM)	Programme und Daten; ohne Strom Inhalt weg
Flash (früher ROM, EPROM)	meist nur gelesen (BIOS, Firmware); behält Inhalt ohne Strom

CACHE UND LOKALITÄT

- CPU wird schneller, Hauptspeicher kommt nicht hinterher; alles auf die CPU wäre zu teuer → kleiner, schneller Cache.
- **Lokalität:** Programme greifen in kurzer Zeit nur auf einen kleinen Teil des Speichers zu; der nächste Befehl liegt meist direkt neben dem letzten → Nachbarn gleich mitladen.
- Beispiele: **Schleifen** (gleicher Programmteil mehrfach), **Arrays/Tabellen** (benachbarte Datensätze vorab holen).
- **Cache-Miss** (Folie: „Cache-fault“): Bereich fehlt im Cache, muss aus dem Hauptspeicher nachgeladen werden.

ADRESSLEITUNGEN

Zelle = 8 Bit (1 Byte), jede mit Adresse. **n Leitungen → 2^n Adressen (Byte).** Jede zusätzliche Leitung verdoppelt.

2^{10} = 1 KB · 2^{20} = 1 MB · 2^{30} = 1 GB · 2^{40} = 1 TB (Faktor 1 024).

Leitungen → Größe:

1. Exponent in 10/20/30/40 + Rest zerlegen.
2. Rest als Zweierpotenz ausrechnen.
3. Mit der Einheit verbinden.

32 = 2 + 30 → 2^2 · 1 GB = **4 GB**. 16 → 64 KB. 24 → 16 MB. 48 → 256 TB.

Größe → Leitungen: Zahl und Einheit als Zweierpotenz schreiben, Exponenten addieren. 512 GB = 2^9 · 2^{30} → **39 Leitungen**. 16 GB = 2^4 · 2^{30} → 34.

SPEICHERHIERARCHIE

```
Register
L1-Cache (kB)
L2-Cache (MB)
L3-Cache
RAM (GB)
——— volatil ↑ / persistent ↓ ———
Flash / SSD (TB)
Festplatte (TB)
Band, optisch, Cloud (Backup)
```

- Nach unten wachsen **Zugriffszeit** (langsamer), **Kapazität** und **Bits pro Euro** (billiger pro Bit).
- **Volatil:** Inhalt ohne Strom weg (Register, Cache, RAM). **Persistent:** bleibt (Flash, SSD, Festplatte, Band).
- **Festplatte:** magnetische Scheiben, Schreib-/Lesekopf. **SSD:** NAND-/NOR-Gatter, keine beweglichen Teile, sehr schnell, begrenzte Schreibzyklen. **Magnetband:** Backup.

RAID

RAID (Redundant Array of Independent Disks): mehrere Platten als ein logisches Laufwerk; Ziel mehr Tempo und/oder Ausfallsicherheit.

- **Striping:** Daten in Stücke zerlegt, reihum verteilt, paralleles Lesen.
- **Mirroring:** Daten auf zweite Platte gespiegelt.

Level	Prinzip	min. Platten	Ausfälle	Nutzkapazit (n Platten à C
0	Striping ohne Redundanz	2	0	n · C
1	Mirroring	2	1 je Spiegelpaar	n · C / 2
5	Striping + 1 Paritätsblock je Zeile, reihum verteilt	3	1	(n − 1) · C
6	wie 5, doppelte Parität	4	2	(n − 2) · C

Nutzkapazität: n und C ablesen, Formel wählen, einsetzen.

	RAID 0	RAID 1	RAID 5	RAID 6
4 × 2 TB	8 TB	4 TB	6 TB	4 TB
5 × 4 TB	20 TB	–	16 TB	12 TB

PARITÄT MIT XOR

1. Parität = alle Datenblöcke spaltenweise XOR (1 bei ungerader Anzahl Einsen).
2. Ausfall: übrige Blöcke und Parität XOR → fehlender Block.

```
1011 ⊕ 0110 ⊕ 1100 = 0001   Parität
Platte 2 fällt aus:
1011 ⊕ 1100 ⊕ 0001 = 0110   wiederhergestellt
```

TYPISCHE FEHLER

- Register zum Hauptspeicher zählen. Sie sitzen in der CPU.
- „Cache lohnt sich, weil er groß ist.“ Er ist klein; Grund ist die Lokalität.
- Adressen als n · 8 oder 2 · n rechnen statt 2^n.
- Grenze volatil/persistent falsch setzen: sie liegt zwischen RAM und Flash/SSD.
- „Nach unten kürzere Zugriffszeit“: sie wird länger.
- RAID 0 für ausfallsicher halten.
- Mindestplatten verwechseln: RAID 5 = 3, RAID 6 = 4.
- RAID für ein Backup halten: Löschen wird sofort auf alle Platten übertragen.
- Folie nennt Flash/SSD „Primärspeicher“; üblich: Primärspeicher = Arbeitsspeicher, SSD/Festplatte = Sekundärspeicher.

PROBLEME DES DATEIANSATZES

1. **Redundanz und Inkonsistenz** (Kopien widersprechen sich) · 2. **Programm-Daten-Abhängigkeit** · 3. **fehlender Datenaustausch**, wenig Flexibilität · 4. **mangelnde Datensicherheit**

DB, DBMS, DATA DICTIONARY

- **Datenbank (DB):** Sammlung logisch zusammenhängender Daten für einen Zweck.
- **DBMS:** Software, mit der man eine DB erstellt und pflegt.
- **Datenbanksystem** = DB + DBMS (+ Anwendungen).
- **Data Dictionary:** Datenbankkatalog mit **Metadaten** („Daten über Daten“): Tabellen, Spalten, Datentypen, Einschränkungen. Daneben die Nutzdaten.
- **Vorteile:** Redundanzkontrolle (Views), Zugriffsbeschränkungen, dauerhafte Speicherung, Mehrbenutzerzugriff, Integritätsbedingungen, Wiederherstellung.
- **Nachteile:** hohe Anfangskosten, für einfache Fälle zu komplex, Mehraufwand für Sicherheit/Wiederherstellung.

ER-MODELL (KONZEPTUELLE EBENE)

Ebenen: konzeptuell (ER) → logisch (Tabellen) → physisch (Dateien, Indizes).

- **Entität:** ein Ding, über das Daten gespeichert werden. **Entitätstyp** (alle gleichartigen) = Rechteck.
- **Attribut** = Ellipse, **Schlüsselattribut** = Ellipse unterstrichen. **Beziehungstyp** = Raute, kann eigene Attribute haben (Rückgabedatum einer Ausleihe).

Kardinalität: 1:1 (Leser – Bibliothekskarte) · 1:n (Rechnung – Posten) · m:n (Student – Vorlesung).

1. Wie viele B hat ein A?
2. Wie viele A hat ein B?
3. Zusammensetzen. Zweimal „viele“ → m:n.

RELATIONALES MODELL

- **Relation** = Tabelle, **Tupel** = Zeile, **Attribut** = Spalte mit Datentyp; Werte atomar.
- **Primärschlüssel:** identifiziert jede Zeile eindeutig. **Fremdschlüssel:** verweist auf den Primärschlüssel einer anderen Tabelle.

ER-Element	wird zu
Entitätstyp	Tabelle, Schlüssel → Primärschlüssel
1:n	Fremdschlüssel auf der **n-Seite**
1:1	Fremdschlüssel auf einer Seite
m:n	Hilfstabelle mit beiden Fremdschlüsseln (+ Beziehungsattribute)

m:n Beispiel: STUDENT(MatrNr, Name), VORLESUNG(VNr, Titel), BESUCHT(MatrNr, VNr).

SQL ÄNDERN (DEKLARATIV: WAS, NICHT WIE)

```
CREATE TABLE Firma (Kurzname char(15) not null, Name
char(40));
INSERT INTO Student (MatrNr, Semester) VALUES
(12345678, 4);
UPDATE Student SET Exmatrikuliert = TRUE WHERE
Semester > 99;
DELETE FROM Student WHERE MatrNr = 87654321;
```

SQL LESEN

Ausführungsreihenfolge: 1. **FROM** (Tabellen, Join) → 2. **WHERE** (Zeile für Zeile) → 3. **GROUP BY** → 4. **SELECT**.

- **Aggregate:** SUM, MAX, MIN, AVG, COUNT. Ohne GROUP BY genau eine Zeile, mit GROUP BY eine je Gruppe.
- **Join:** alle Tabellen ins FROM, „Fremdschlüssel = Primärschlüssel“ ins WHERE.

```
SELECT PNUMMER, ABTNR, NNAME
FROM PROJEKT, ABTEILUNG, ANGESTELLTER
WHERE ABTNR = ABTNUMMER AND MGRSSN = SSN
  AND PSTANDORT = 'Stafford';
```

Projekte 10, 30 → Abteilung 4 → MGRSSN 987654321 → Wallace: **10 | 4 | Wallace**, **30 | 4 | Wallace**.

GROUP BY ABT mit AVG(GEHALT): Abt. 4 = (25000 + 43000 + 25000) / 3 = **31000**.

SQL SCHREIBEN

1. Was ausgeben? → SELECT; „wie viele/Summe/Durchschnitt/höchstes“ → Aggregat.
2. Wo stehen die Spalten? → FROM, bei mehreren Tabellen Join ins WHERE.
3. Welche Zeilen? → WHERE mit AND/OR, Texte in '…'.
4. „pro“/„je“ → GROUP BY. „Projekte je Abteilung“: `SELECT ABTNR, COUNT(*) FROM PROJEKT GROUP BY ABTNR;`

OLTP UND OLAP

- **OLTP:** Tagesgeschäft (Bestellung, Buchung), aktuelle Daten, ganze Datensätze lesen/schreiben, Row Store (zeilenweise); Oracle, MySQL.
- **OLAP:** Analyse (BI, Data Warehouse), mit Historie, fast nur lesen, Aggregate, Column Store (spaltenweise); Amazon Redshift, SAP HANA.

ETL (Extract, Transform, Load) bringt Daten von OLTP nach OLAP.

TYPISCHE FEHLER

- Redundanz allein als Problem nennen; das Problem ist die Inkonsistenz.
- DBMS (Software) und Datenbank (Datenbestand) vertauschen.
- Kardinalität nur aus einer Richtung fragen.
- Fremdschlüssel bei 1:n auf die 1-Seite setzen.
- UPDATE/DELETE ohne WHERE → alle Zeilen betroffen.
- Typografische statt gerader Hochkommas; DELETE ohne FROM.
- `>` und `>=` verwechseln (genau 30000 fällt bei `> 30000` heraus); AND und OR verwechseln.
- Aggregat ohne GROUP BY, aber mehrere Zeilen erwarten.
- Join-Bedingung vergessen → jede Zeile mit jeder kombiniert.
- Data Warehouse als OLTP-System einordnen.